Logistic Regression

Jyotishka Datta

Updated 2025-02-19

The Stock Market Data

We will begin by examining some numerical and graphical summaries of the `Smarket` data, which is part of the `ISLR` library.

This data set consists of percentage returns for the S & P 500 stock index over 1, 250 days, from the beginning of 2001 until the end of 2005. For each date, we have recorded the percentage returns for each of the five previous trading days, `Lag1` through `Lag5`.

We have also recorded `Volume` (the number of shares traded on the previous day, in billions), `Today` (the percentage return on the date in question) and `Direction` (whether the market was `Up` or `Down` on this date).

Smarket

```
library(ISLR)
```

```
## Warning: package 'ISLR' was built under R version 4.4.2
```

```
names(Smarket)
```

```
## [1] "Year"      "Lag1"      "Lag2"      "Lag3"      "Lag4"      "Lag5"
## [7] "Volume"    "Today"     "Direction"
```

```
dim(Smarket)
```

```
## [1] 1250    9
```

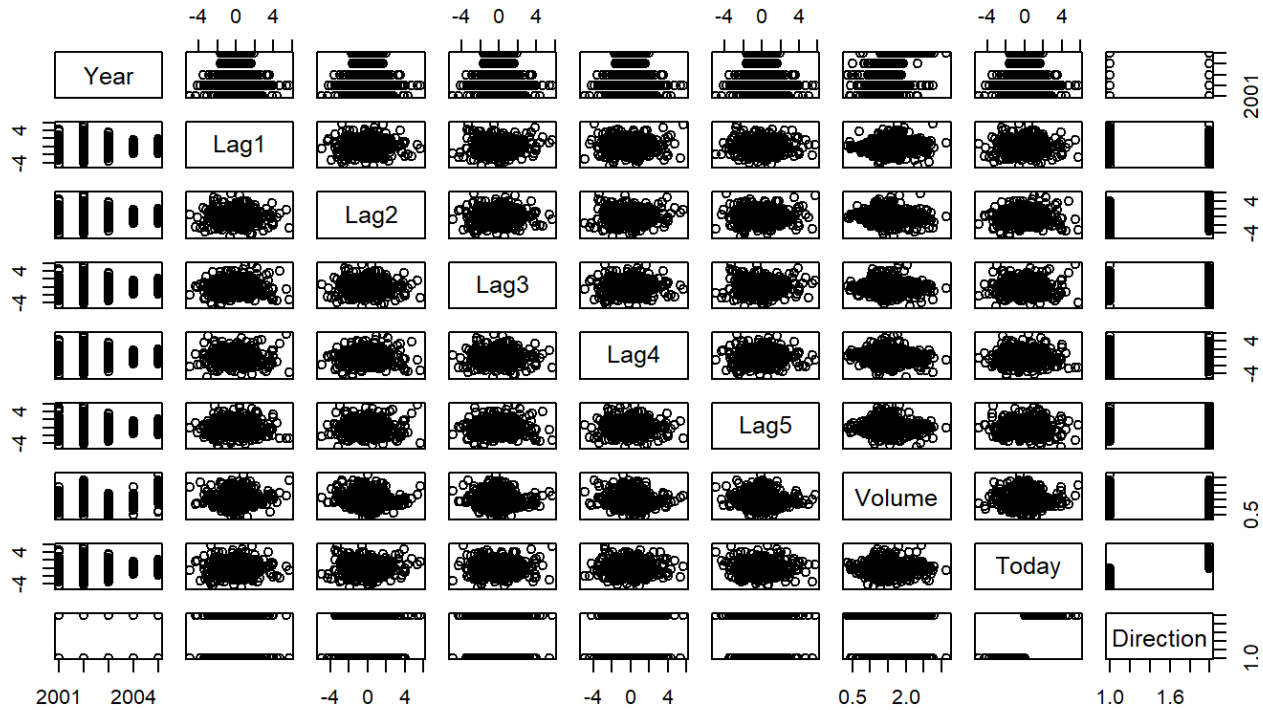
Smarket

```
summary(Smarket)
```

```
##       Year           Lag1                Lag2                Lag3
##  Min.   :2001   Min.   :-4.922000   Min.   :-4.922000   Min.   :-4.922000
##  1st Qu.:2002   1st Qu.:-0.639500   1st Qu.:-0.639500   1st Qu.:-0.640000
##  Median :2003   Median : 0.039000   Median : 0.039000   Median : 0.038500
##  Mean   :2003   Mean   : 0.003834   Mean   : 0.003919   Mean   : 0.001716
##  3rd Qu.:2004   3rd Qu.: 0.596750   3rd Qu.: 0.596750   3rd Qu.: 0.596750
##  Max.   :2005   Max.   : 5.733000   Max.   : 5.733000   Max.   : 5.733000
##       Lag4                Lag5              Volume           Today
##  Min.   :-4.922000   Min.   :-4.92200   Min.   :0.3561   Min.   :-4.922000
##  1st Qu.:-0.640000   1st Qu.:-0.64000   1st Qu.:1.2574   1st Qu.:-0.639500
##  Median : 0.038500   Median : 0.03850   Median :1.4229   Median : 0.038500
##  Mean   : 0.001636   Mean   : 0.00561   Mean   :1.4783   Mean   : 0.003138
##  3rd Qu.: 0.596750   3rd Qu.: 0.59700   3rd Qu.:1.6417   3rd Qu.: 0.596750
##  Max.   : 5.733000   Max.   : 5.73300   Max.   :3.1525   Max.   : 5.733000
##  Direction
##  Down:602
##  Up  :648
##
##
##
##
```

Smarket

```
pairs(Smarket)
```



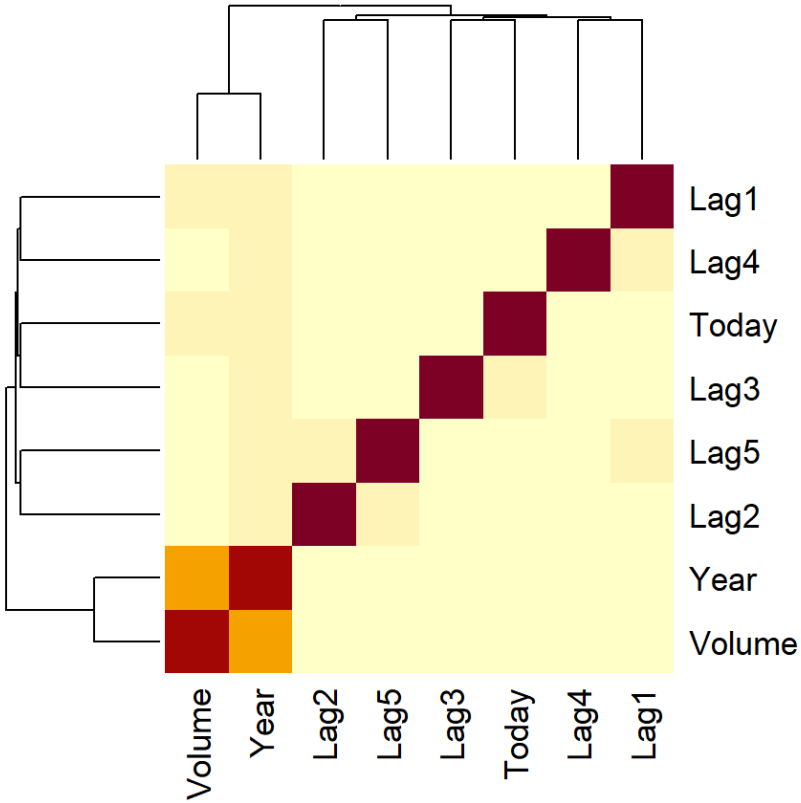
Smarket

```
cor(Smarket[,-9])
```

```
##              Year         Lag1         Lag2         Lag3         Lag4
## Year   1.00000000  0.029699649  0.030596422  0.033194581  0.035688718
## Lag1   0.02969965  1.000000000 -0.026294328 -0.010803402 -0.002985911
## Lag2   0.03059642 -0.026294328  1.000000000 -0.025896670 -0.010853533
## Lag3   0.03319458 -0.010803402 -0.025896670  1.000000000 -0.024051036
## Lag4   0.03568872 -0.002985911 -0.010853533 -0.024051036  1.000000000
## Lag5   0.02978799 -0.005674606 -0.003557949 -0.018808338 -0.027083641
## Volume 0.53900647  0.040909908 -0.043383215 -0.041823686 -0.048414246
## Today  0.03009523 -0.026155045 -0.010250033 -0.002447647 -0.006899527
##                Lag5      Volume        Today
## Year    0.029787995  0.53900647  0.030095229
## Lag1   -0.005674606  0.04090991 -0.026155045
## Lag2   -0.003557949 -0.04338321 -0.010250033
## Lag3   -0.018808338 -0.04182369 -0.002447647
## Lag4   -0.027083641 -0.04841425 -0.006899527
## Lag5    1.000000000 -0.02200231 -0.034860083
## Volume -0.022002315  1.00000000  0.014591823
## Today  -0.034860083  0.01459182  1.000000000
```

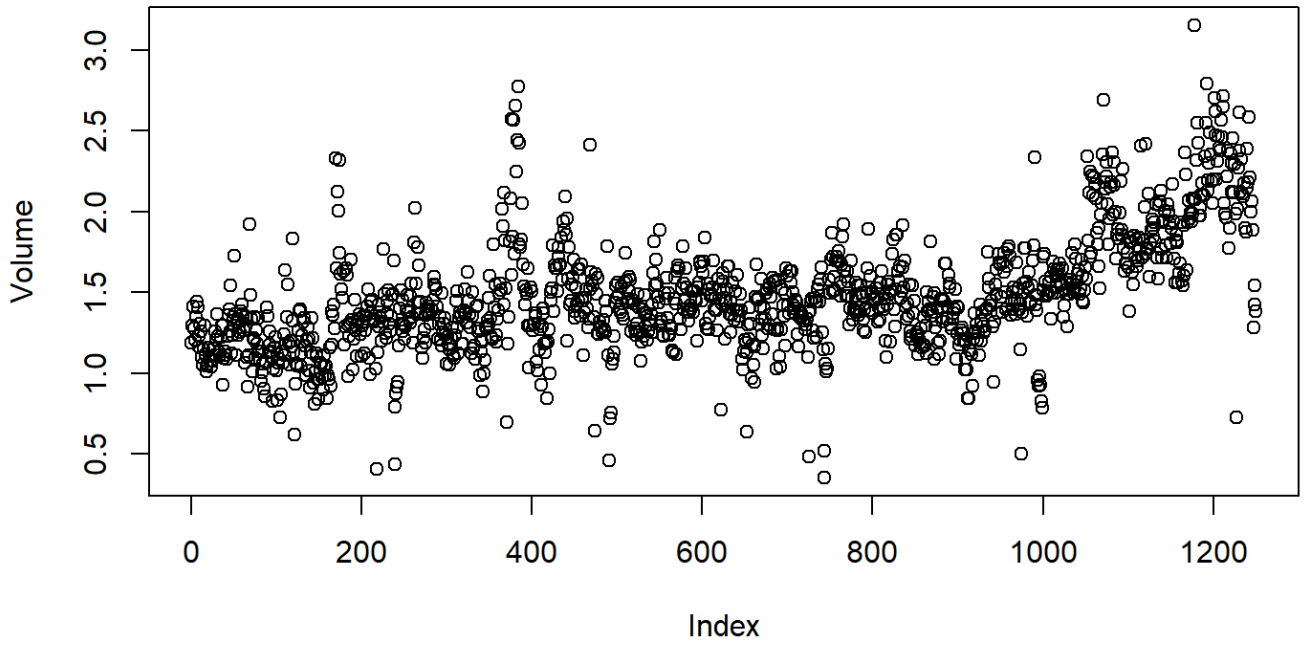
Smarket

```
heatmap(cor(Smarket[,-9]))
```



Smarket

```
attach(Smarket)
plot(Volume)
```



Logistic Regression

Next, we will fit a logistic regression model in order to predict `Direction` using `Lag1` through `Lag5` and `Volume`.

The `glm()` function fits generalized linear models, a class of models that includes logistic regression.

The syntax of the `glm()` function is similar to that of `lm()`, except that we must pass in linear model the argument `family=binomial` in order to tell `R` to run a logistic regression rather than some other type of generalized linear model.

Logistic Regression

```
glm.fits=glm(Direction~Lag1+Lag2+Lag3+Lag4+Lag5+Volume,data=Smarket,family=binomial)
summary(glm.fits)
```

```
##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 +
##     Volume, family = binomial, data = Smarket)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.126000   0.240736  -0.523    0.601
## Lag1        -0.073074   0.050167  -1.457    0.145
## Lag2        -0.042301   0.050086  -0.845    0.398
## Lag3         0.011085   0.049939   0.222    0.824
## Lag4         0.009359   0.049974   0.187    0.851
## Lag5         0.010313   0.049511   0.208    0.835
## Volume       0.135441   0.158360   0.855    0.392
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1731.2  on 1249  degrees of freedom
## Residual deviance: 1727.6  on 1243  degrees of freedom
## AIC: 1741.6
##
## Number of Fisher Scoring iterations: 3
```

Logistic Regression

```
coef(glm.fits)
```

```
##  (Intercept)         Lag1         Lag2         Lag3         Lag4         Lag5
## -0.126000257 -0.073073746 -0.042301344  0.011085108  0.009358938  0.010313068
##       Volume
##  0.135440659
```

```
summary(glm.fits)$coef
```

```
##                 Estimate Std. Error    z value  Pr(>|z|)
## (Intercept) -0.126000257 0.24073574 -0.5233966 0.6006983
## Lag1        -0.073073746 0.05016739 -1.4565986 0.1452272
## Lag2        -0.042301344 0.05008605 -0.8445733 0.3983491
## Lag3         0.011085108 0.04993854  0.2219750 0.8243333
## Lag4         0.009358938 0.04997413  0.1872757 0.8514445
## Lag5         0.010313068 0.04951146  0.2082966 0.8349974
## Volume       0.135440659 0.15835970  0.8552723 0.3924004
```

Logistic Regression

```
summary(glm.fits)$coef[,4]
```

```
## (Intercept)        Lag1        Lag2        Lag3        Lag4        Lag5
##   0.6006983   0.1452272   0.3983491   0.8243333   0.8514445   0.8349974
##      Volume
##   0.3924004
```

Prediction

The `predict()` function can be used to predict the probability that the market will go up, given values of the predictors.

The `type="response"` option tells `R` to output probabilities of the form $P(Y = 1|X)$.

If no test data set is supplied to the `predict()` function, then the probabilities are computed for the training data that was used to fit the logistic regression model.

Prediction

```
glm.probs=predict(glm.fits,type="response")
glm.probs[1:10]
```

```
##         1         2         3         4         5         6         7         8
## 0.5070841 0.4814679 0.4811388 0.5152224 0.5107812 0.5069565 0.4926509 0.5092292
##         9        10
## 0.5176135 0.4888378
```

Up or Down?

The `contrasts()` function indicates that R has created a dummy variable with a 1 for Up.

```
contrasts(Direction)
```

```
##      Up
## Down  0
## Up    1
```

Predictions

_ Must convert these predicted probabilities into class labels, `Up` or `Down`.

- Can create a vector of class predictions based on whether the predicted probability of a market increase is greater than or less than 0.5.

```
##         Direction
## glm.pred Down  Up
##     Down  145 141
##     Up    457 507
```

Confusion Matrix

```
(507+145)/1250
```

```
## [1] 0.5216
```

```
mean(glm.pred==Direction)
```

```
## [1] 0.5216
```

52% : Little better than random guessing.

Still this is misleading - training error can be overly optimistic !

Supervised Learning - Predicting the future Stock

Training Data: Before 2005, Test data: after 2005.

```
train=(Year<2005)
Smarket.2005=Smarket[!train,]
dim(Smarket.2005)
```

```
## [1] 252   9
```

```
Direction.2005=Direction[!train]
glm.fits=glm(Direction~Lag1+Lag2+Lag3+Lag4+Lag5+Volume,data=Smarket,family=binomial,subset=train)
glm.probs=predict(glm.fits,Smarket.2005,type="response")
```

Supervised Learning

```
glm.pred=rep("Down",252)
glm.pred[glm.probs>.5]="Up"
table(glm.pred,Direction.2005)
```

```
##         Direction.2005
## glm.pred Down Up
##     Down   77 97
##     Up     34 44
```

```
mean(glm.pred==Direction.2005)
```

```
## [1] 0.4801587
```

- 48% - Worse than random guessing !

Prediction improves with better modelling

```
glm.fits=glm(Direction~Lag1+Lag2,data=Smarket,family=binomial,subset=train)
glm.probs=predict(glm.fits,Smarket.2005,type="response")
glm.pred=rep("Down",252)
glm.pred[glm.probs>.5]="Up"
table(glm.pred,Direction.2005)
```

```
##         Direction.2005
## glm.pred Down  Up
##     Down   35  35
##     Up     76 106
```

```
mean(glm.pred==Direction.2005)
```

```
## [1] 0.5595238
```

Separability

- The usual method of fitting a Logistic Regression doesn't work if one of the predictors perfectly separate the response.
- The MLE does not exist in that situation.
- Why?

Separability

- Generate data: x_1 is unrelated, x_2 perfectly separates y.

```
set.seed(123456)
d <- data.frame(y  =  c(0,0,0,0,0, 1,1,1,1,1),
                x1 = rnorm(10),
                x2 = c(runif(5,-3,0),runif(5,0,3)))

fit <- glm(y ~ x1 + x2, data=d, family="binomial")
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

What happens to the estimates?

```
##
## Call:
## glm(formula = y ~ x1 + x2, family = "binomial", data = d)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)    11.77   61207.76       0        1
## x1            -10.44   79869.77       0        1
## x2             15.04   36190.44       0        1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1.3863e+01  on 9  degrees of freedom
## Residual deviance: 2.3800e-10  on 7  degrees of freedom
## AIC: 6
##
## Number of Fisher Scoring iterations: 24
```

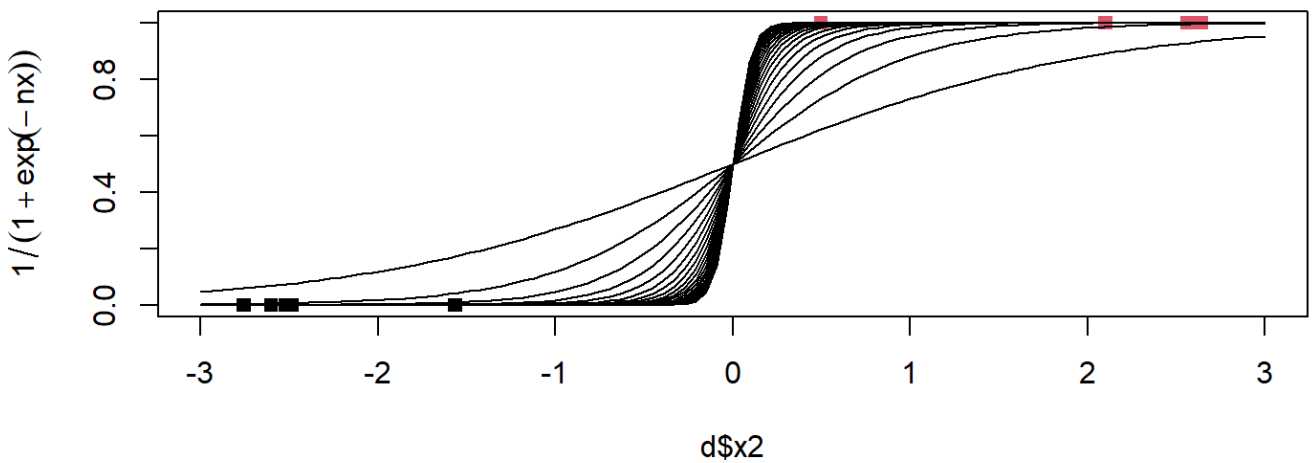
Why?

- Classification rule based on probability:

$$P(Y = 0) = 1/(1 + \exp(-\beta x))$$

- Figure below is the sequence of curves:

$$1/(1 + \exp(-x)), 1/(1 + \exp(-2x)), 1/(1 + \exp(-3x)), \ldots, 1/(1 + \exp(-nx))$$



Solution

- The “best” solution is to use a Cauchy prior on each coefficient.
- Gelman (2008): Cauchy(0, 2.5) for variance parameters.
- Intuitively, Cauchy has heavier tails and it allows large values and also it shrinks the β’s just the right amount.
- Prior distributions for variance parameters in hierarchical models (http://www.stat.columbia.edu/~gelman/research/published/taumain.pdf)
- This is easy enough for you to write your own sampling algorithm.

The `arm` package on R

```
library(arm)
fit <- bayesglm(y ~ x1 + x2, data=d, family="binomial")
display(fit)
```

```
## bayesglm(formula = y ~ x1 + x2, family = "binomial", data = d)
##             coef.est coef.se
## (Intercept)  0.55     1.40
## x1          -0.21     0.78
## x2           1.32     0.58
## ---
## n = 10, k = 3
## residual deviance = 1.4, null deviance = 13.9 (difference = 12.4)
```